Assignment 4 extra solution

1. Q1

- **ALU Testbench**

```
module ALU_tb (
  //? Interface signals
  input logic clk,
  output logic reset,
  output logic signed [3:0] A, B,
  input logic signed [4:0] C,
  output logic [1:0] opcode
);
  //? parameters
  localparam MAXPOS = 7;
  localparam MAXNEG = -8;
  localparam Add = 2'b00;
  localparam Sub = 2'b01;
  localparam Not_A = 2'b10;
  localparam ReductionOR_B = 2'b11;
  localparam CLK_CYCLE = 2;
  //? Driver block
  initial begin
    //! ALU_1 test
    reset = 0;
    A = $random;
    B = $random;
    opcode = $random;
    #(CLK_CYCLE)
    reset = 1;
    #CLK_CYCLE;
    reset = 0;
    //! ALU_2 test
    //op-code iterator
    for (int i = 0; i < 2; i++) begin
      if (i == 0) opcode = Add;
      else opcode = Sub;
      //input A iterator
      for (int j = 0; j < 3; j++) begin
        if (j == 0) A = 0;
        else if (j == 1) A = MAXPOS;
        else A = MAXNEG;
        //input B iterator
        for (int k = 0; k < 3; k++) begin
          if (k == 0) B = 0;
          else if (k == 1) B = MAXPOS;
          else B = MAXNEG;
          #CLK_CYCLE;
        end
      end
    end
    //! ALU_3 test
```

```
      //! ALU_3 test
      B = $random;
      A = 4'b1111;
      opcode = Not_A;
      #CLK_CYCLE
      A = 4'b0000;
      #CLK_CYCLE
      A = 4'b0001;
      #CLK_CYCLE
      A = 4'b0010;
      #CLK_CYCLE
      A = 4'b0100;
      #CLK_CYCLE
      A = 4'b1000;
      #CLK_CYCLE
      //! ALU_4 test
      A = $random;
      B = 4'b1111;
      opcode = ReductionOR_B;
      #CLK_CYCLE
      B = 4'b0000;
      #CLK_CYCLE
      B = 4'b0001;
      #CLK_CYCLE
      B = 4'b0010;
      #CLK_CYCLE
      B = 4'b0100;
      #CLK_CYCLE
      B = 4'b1000;
      #CLK_CYCLE
      //! ALU_5 test
      for(int i=0; i<100; i++)begin
        A = $random;
        B = $random;
        opcode = $random;
        if(i<10) begin
          reset = 1;
          #CLK_CYCLE;
        end
        else reset = 0;
      end
      $stop;
    end
  endmodule
```

- **TOP**

```
module TOP ();
  logic reset;
  logic signed [3:0] A, B;
  logic signed [4:0] C;
  logic [1:0] opcode;
  bit clk;
  always #1 clk = ~clk;

  ALU_4_bit DUT (.reset(reset), .A(A), .B(B), .C(C), .opcode(opcode), .clk(clk));
  ALU_tb TB (.reset(reset), .A(A), .B(B), .C(C), .opcode(opcode), .clk(clk));
  bind ALU_4_bit ALU_sva ALU_sva_inst (.reset(reset), .A(A), .B(B), .C(C), .opcode(opcode), .clk(clk));
endmodule
```

- **SVA file**

```
module ALU_sva(
    input logic clk,
    input logic reset,
    input logic signed [3:0] A, B,
    input logic signed [4:0] C,
    input logic [1:0] opcode
);
    localparam Add = 2'b00;
    localparam Sub = 2'b01;
    localparam Not_A = 2'b10;
    localparam ReductionOR_B = 2'b11;
    //? asynchronous reset assertion
    always_comb begin
        if(reset)
            arst_sva: assert final(C == 0);
    end
    //? PROPERTIES
    property add_sva_prop;
        @(posedge clk) disable iff(reset) opcode == Add |=> C == $past(A) + $past(B);
    endproperty
    property sub_sva_prop;
        @(posedge clk) disable iff(reset) opcode == Sub |=> C == $past(A) - $past(B);
    endproperty
    property not_a_sva_prop;
        @(posedge clk) disable iff(reset) opcode == Not_A |=> C == ~$past(A);
    endproperty
    property reduction_sva_prop;
        @(posedge clk) disable iff(reset) opcode == ReductionOR_B |=> C == |$past(B);
    endproperty
    //? ASSERTIONS
    add_sva:       assert property (add_sva_prop);
    sub_sva:       assert property (sub_sva_prop);
    not_a_sva:     assert property (not_a_sva_prop) ;
    reduction_sva: assert property (reduction_sva_prop);
    //? COVER PROPERTIES
    add_sva_cp:       cover property (add_sva_prop);
    sub_sva_cp:       cover property (sub_sva_prop);
    not_a_sva_cp:     cover property (not_a_sva_prop) ;
    reduction_sva_cp: cover property (reduction_sva_prop);
endmodule
```

- **Assertion results and coverage**

Name	Assertion Type	Language	Enable	Failure Count	Pass Count	A
/TOP/DUT/ALU_sva_inst/arst_sva	Immediate	SVA	on	0	1	
/TOP/DUT/ALU_sva_inst/add_sva	Concurrent	SVA	on	0	1	
/TOP/DUT/ALU_sva_inst/sub_sva	Concurrent	SVA	on	0	1	
/TOP/DUT/ALU_sva_inst/not_a_sva	Concurrent	SVA	on	0	1	
/TOP/DUT/ALU_sva_inst/reduction_sva	Concurrent	SVA	on	0	1	

/TOP/DUT/ALU_sva_inst/add_sva_cp	SVA	✓	Off	9	1	Unlimited	1	100%		✓
/TOP/DUT/ALU_sva_inst/sub_sva_cp	SVA	✓	Off	9	1	Unlimited	1	100%		✓
/TOP/DUT/ALU_sva_inst/not_a_sva_cp	SVA	✓	Off	6	1	Unlimited	1	100%		✓
/TOP/DUT/ALU_sva_inst/reduction_sva_cp	SVA	✓	Off	5	1	Unlimited	1	100%		✓

- **Code coverage**

❖ **Statement**

```
ALU.v
✓   20 always @* begin
✓   22 Add:            Alu_out = A + B;
✓   23 Sub:            Alu_out = A - B;
✓   24 Not_A:          Alu_out = ~A;
✓   25 ReductionOR_B:  Alu_out = |B;
E   26 default:  Alu_out = 5'b0;  //! execlude this li
✓   31 always @(posedge clk or posedge reset) begin
✓   33 C <= 5'b0;
✓   35 C<= Alu_out;
```

❖ **Branch**

```
ALU.v
✓   22 Add:            Alu_out = A + B;
✓   23 Sub:            Alu_out = A - B;
✓   24 Not_A:          Alu_out = ~A;
✓   25 ReductionOR_B:  Alu_out = |B;
E   26 default:  Alu_out = 5'b0;  //! exe
✓   32 if (reset)
✓   34 else
```

❖ **Toggle**

```
sim:/TOP/DUT
  ✓ A
  ✓ Alu_out
  ✓ B
  ✓ C
  ✓ clk
  ✓ opcode
  ✓ reset
```

- **Priority Encoder testbench**

```
module priority_enc_tb (
  //?interface ports
  output logic [3:0] D,
  output logic rst,
  input logic clk,
  input logic [1:0] Y,
  input logic valid
);
  localparam CLK_CYCLE = 2;
  //?test block
  initial begin
    D   = 0;
    rst = 0;
    #(CLK_CYCLE / 2);
    assert_reset();
    assert_data();
    assert_reset();
    $stop;
  end
  //?tasks
  task assert_reset();
    rst = 1;
    #(5*CLK_CYCLE);
    rst = 0;
  endtask
  task assert_data();
    for (int i = 0; i < 17; i++) begin
      D++;
      #CLK_CYCLE;
    end
  endtask

endmodule
```

- **TOP**

```
module TOP ();
  logic rst, valid;
  logic [3:0] D;
  logic [1:0] Y;
  bit clk;
  always #1 clk = ~clk;

  priority_enc PE_DUT (.D(D), .clk(clk), .rst(rst), .Y(Y), .valid(valid));
  priority_enc_tb TB (.D(D), .clk(clk), .rst(rst), .Y(Y), .valid(valid));
  bind priority_enc pencoder_sva pencoder_sva_inst (.D(D), .clk(clk), .rst(rst), .Y(Y), .valid(valid));
endmodule
```

- **SVA file**

```
module pencoder_sva(
    input logic clk,
    input logic rst,
    input logic [3:0] D,
    input logic [1:0] Y,
    input logic valid
);
    //? PROPERTIES
    property reset_sva_prop;
        @(posedge clk) rst |=> $past(Y) == 0 && $past(valid) == 0;
    endproperty
    property D0_sva_prop;
        @(posedge clk) disable iff(rst) D == 4'b1000 |=> $past(Y) == 0;
    endproperty
    property D1_sva_prop;
        @(posedge clk) disable iff(rst) !D[0] && !D[1] && D[2] |=> $past(Y) == 1;
    endproperty
    property D2_sva_prop;
        @(posedge clk) disable iff(rst) !D[0] && D[1] |=> $past(Y) == 2;
    endproperty
    property D3_sva_prop;
        @(posedge clk) disable iff(rst) D[0] |=> $past(Y) == 3;
    endproperty
    property not_valid_sva_prop;
        @(posedge clk) disable iff(rst) ~|D |=> $past(valid) == 0;
    endproperty
    property valid_sva_prop;
        @(posedge clk) disable iff(rst)  |D |=> $past(valid) == 1;
    endproperty
    //? ASSERTIONS
    reset_sva:     assert property (reset_sva_prop);
    D0_sva:        assert property (D0_sva_prop);
    D1_sva:        assert property (D1_sva_prop) ;
    D2_sva:        assert property (D2_sva_prop);
    D3_sva:        assert property (D2_sva_prop);
    not_valid_sva: assert property (not_valid_sva_prop);
    valid_sva:     assert property (valid_sva_prop);
    //? COVER PROPERTIES
    reset_sva_cp:     cover property (reset_sva_prop);
    D0_sva_cp:        cover property (D0_sva_prop);
    D1_sva_cp:        cover property (D1_sva_prop) ;
    D2_sva_cp:        cover property (D2_sva_prop);
    D3_sva_cp:        cover property (D3_sva_prop);
    not_valid_sva_cp: cover property (not_valid_sva_prop);
    valid_sva_cp:     cover property (valid_sva_prop);
endmodule
```

- **Assertion results and coverage**

Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Acti
/TOP/PE_DUT/pencoder_sva_inst/reset_sva	Concurrent	SVA	on	0	1	
/TOP/PE_DUT/pencoder_sva_inst/D0_sva	Concurrent	SVA	on	0	1	
/TOP/PE_DUT/pencoder_sva_inst/D1_sva	Concurrent	SVA	on	0	1	
/TOP/PE_DUT/pencoder_sva_inst/D2_sva	Concurrent	SVA	on	0	1	
/TOP/PE_DUT/pencoder_sva_inst/D3_sva	Concurrent	SVA	on	0	1	
/TOP/PE_DUT/pencoder_sva_inst/not_valid_sva	Concurrent	SVA	on	0	1	
/TOP/PE_DUT/pencoder_sva_inst/valid_sva	Concurrent	SVA	on	0	1	

Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Includ
/TOP/PE_DUT/pencoder_sva_inst/reset_sva_cp	SVA	✓	Off	8	1	Unlimited	1	100%		✓
/TOP/PE_DUT/pencoder_sva_inst/D0_sva_cp	SVA	✓	Off	1	1	Unlimited	1	100%		✓
/TOP/PE_DUT/pencoder_sva_inst/D1_sva_cp	SVA	✓	Off	2	1	Unlimited	1	100%		✓
/TOP/PE_DUT/pencoder_sva_inst/D2_sva_cp	SVA	✓	Off	4	1	Unlimited	1	100%		✓
/TOP/PE_DUT/pencoder_sva_inst/D3_sva_cp	SVA	✓	Off	8	1	Unlimited	1	100%		✓
/TOP/PE_DUT/pencoder_sva_inst/not_valid_sva_cp	SVA	✓	Off	1	1	Unlimited	1	100%		✓
/TOP/PE_DUT/pencoder_sva_inst/valid_sva_cp	SVA	✓	Off	15	1	Unlimited	1	100%		✓

- **Code coverage**

❖ **Statement**

Statements - by instance (/TOP/PE_DUT)

```
priority_enc.v
 9 always @(posedge clk) begin
11 Y <= 2'b0;
12 valid <= 1'b0;
15 4'b1000: Y <= 0;
16 4'bX100: Y <= 1;
17 4'bXX10: Y <= 2;
18 4'bXXX1: Y <= 3;
20 valid <= (~|D) ? 1'b0 : 1'b1;
```

❖ **Branch**

Branches - by instance (/TOP/PE_DUT)

```
priority_enc.v
10 if (rst) begin
13 end else begin
14 casex (D)
15 4'b1000: Y <= 0;
16 4'bX100: Y <= 1;
17 4'bXX10: Y <= 2;
18 4'bXXX1: Y <= 3;
```

❖ **Toggle**

Toggles - by instance (/TOP/PE_DUT)

```
sim:/TOP/PE_DUT
  clk
  D
  rst
  valid
  Y
```

2. Q2

```
module Q2_sva(
    input logic clk,
    input logic request,
    input logic grant,
    input logic frame,
    input logic irdy
);
    //? PROPERTIES
    property request_sva_prop;
        @(posedge clk) $rose(request) |-> ##[2:5] $rose(grant);
    endproperty
    property grant_sva_prop;
        @(posedge clk) $rose(grant) |-> $fell(frame) && $fell(irdy);
    endproperty
    property complete_sva;
        @(posedge clk) $rose(frame) && $rose(irdy) |=> $fell(grant);
    endproperty
endmodule
```

3. Q3

```
module Q3_sva(
    input logic clk,
    input logic get_data
);
    localparam IDLE = 0;
    localparam GEN_BLK_ADDR = 1;
    localparam WAITO = 2;
    logic cs;
    //? PROPERTIES
    always_comb begin
        assert ($onehot(cs));
    end
    property state_transition;
        @(posedge clk) cs==IDLE && $rose(get_data) |=> cs==GEN_BLK_ADDR ##64 cs==WAITO;
    endproperty
endmodule
```